

Blackthorn

Security Review For Velodrome

Collaborative Audit Prepared For: **Velodrome**

Lead Security Expert(s): **0xdeadbeef**

deadmanwalking

defsec

sammy

Date Audited: **November 3 - November 11, 2025**

Introduction

Running a Safe with high-value funds still depends heavily on off-chain signatures and off-chain payloads. That means signers can see different data, payloads can drift or be spoofed, and stale signatures can be reused.

Canon Guard is a Safe Guard that moves intent on-chain, eliminating external dependencies, so you just need to trust the blockchain.

Canon Guard is designed to keep your Safe actually safe, even if the UI, your Ledger, AWS, and every signer get compromised, all at once.

Scope

Repository: `defi-wonderland/canon-guard`

Audited Commit: `7c26aed5e0210ba88b5527a0fe9dd5884dbb2942`

Final Commit: `8416853ec46ee1b12a565a59e2fc59a42934a608`

Files:

- `script/Constants.sol`
- `script/DeployCanonGuard.s.sol`
- `src/contracts/action-hubs/ActionHubChild.sol`
- `src/contracts/action-hubs/ActionHub.sol`
- `src/contracts/action-hubs/CappedTokenTransfersHub.sol`
- `src/contracts/actions-builders/ActionsBuilder.sol`
- `src/contracts/actions-builders/AllowanceClaimor.sol`
- `src/contracts/actions-builders/CappedTokenTransfers.sol`
- `src/contracts/actions-builders/ChangeSafeGuardAction.sol`
- `src/contracts/actions-builders/EverclearTokenConversion.sol`
- `src/contracts/actions-builders/OPxAction.sol`

- `src/contracts/actions-builders/PreApproveAction.sol`
- `src/contracts/actions-builders/SetEmergencyCallerAction.sol`
- `src/contracts/actions-builders/SetEmergencyTriggerAction.sol`
- `src/contracts/actions-builders/SetGuardAction.sol`
- `src/contracts/actions-builders/SimpleActions.sol`
- `src/contracts/actions-builders/SimpleTransfers.sol`
- `src/contracts/actions-builders/UnsetEmergencyModeAction.sol`
- `src/contracts/CanonGuard.sol`
- `src/contracts/EmergencyModeHook.sol`
- `src/contracts/factories/AllowanceClaimorFactory.sol`
- `src/contracts/factories/CanonGuardFactory.sol`
- `src/contracts/factories/CappedTokenTransfersHubFactory.sol`
- `src/contracts/factories/ChangeSafeGuardActionFactory.sol`
- `src/contracts/factories/EverclearTokenConversionFactory.sol`
- `src/contracts/factories/Factory.sol`
- `src/contracts/factories/OPxActionFactory.sol`
- `src/contracts/factories/PreApproveActionFactory.sol`
- `src/contracts/factories/SetEmergencyCallerActionFactory.sol`
- `src/contracts/factories/SetEmergencyTriggerActionFactory.sol`
- `src/contracts/factories/SimpleActionsFactory.sol`
- `src/contracts/factories/SimpleTransfersFactory.sol`
- `src/contracts/OnlyCanonGuard.sol`
- `src/contracts/SafeManageable.sol`

Final Commit Hash

8416853ec46ee1b12a565a59e2fc59a42934a608

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	17

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0xdeadbeef

A handwritten signature in white ink that reads "0xDeadbeef". The "0x" is written in a stylized, cursive font, and "Deadbeef" follows in a similar script.

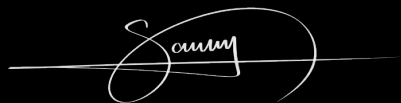
@deadmanwalking

A handwritten signature in white ink that reads "Deadman Walking". The words are written in a cursive, flowing script.

@defsec

A handwritten signature in white ink that reads "Defsec". The letters are written in a large, elegant, cursive script.

@sammy

A handwritten signature in white ink that reads "Sammy". The signature is written in a cursive script with a long horizontal line extending to the left.

Issue L-1: Removed Safe owner can still cancel queued transactions [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/34>

Summary

Removed Safe owner can still cancel a queued transaction they proposed before removal. Cancellation authorization only checks equality with the originally stored proposer, and does not re-validate that the proposer is still a current Safe owner.

Vulnerability Detail

At queue time, the contract stores `msg.sender` as the transaction proposer. At cancel time, the contract allows cancellation if `msg.sender == proposer` (unless in emergency mode) without checking whether that address remains a Safe owner. Safe owners can be removed in the interim via Safe governance (e.g., `removeOwner`, `swapOwner`), so a removed owner retains the ability to cancel the queued transaction.

```
function cancelEnqueuedTransaction(address _actionsBuilder) external {
    _onBeforeExecution();

    TransactionInfo memory _txInfo = transactionsInfo[_actionsBuilder];
    if (_txInfo.expiresAt == 0) revert NoTransactionQueued();
    if (!emergencyMode && msg.sender != _txInfo.proposer) revert
        CallerMustBeTransactionProposer();
```

```
transactionsInfo[_actionsBuilder] = TransactionInfo({
    proposer: msg.sender,
    actionsData: abi.encode(_actions),
    executableAt: block.timestamp + _txExecutionDelay,
    expiresAt: block.timestamp + _txExecutionDelay + TX_EXPIRY_DELAY,
    isPreApproved: _actionIsPreApproved
});
```

Because `cancelEnqueuedTransaction` does not ensure the proposer is still a Safe owner, any previously-removed owner can still cancel the transaction they proposed earlier.

Proof Of Concept

```
function test_PoC_RemovedOwnerCanStillCancel(
    address _proposer,
    address _actionsBuilder,
    address _target,
    uint256 _value,
    bytes memory _data
) external {
    _assumeFuzzable(_actionsBuilder);

    // Step 1: Proposer is a Safe owner and queues a transaction
    _mockAndExpect(SAFE, abi.encodeWithSelector(IOwnerManager.isOwner.selector),
        abi.encode(true));
    vm.mockCallRevert(_actionsBuilder,
        abi.encodeWithSelector(IActionHubChild.HUB.selector), 'Not implemented');

    IActionsBuilder.Action[] memory _actions = new IActionsBuilder.Action[](1);
    _actions[0] = IActionsBuilder.Action({target: _target, value: _value, data:
        _data});
    _mockAndExpect(
        address(_actionsBuilder),
        abi.encodeWithSelector(IActionsBuilder.getActions.selector),
        abi.encode(_actions)
    );

    vm.prank(_proposer);
    canonGuard.queueTransaction(_actionsBuilder);

    // Step 2: Proposer is removed from Safe owners (no longer an owner)
    // Note: cancelEnqueuedTransaction does not check ownership, so don't expect a
    // call here.
    vm.mockCall(SAFE, abi.encodeWithSelector(IOwnerManager.isOwner.selector),
        abi.encode(false));

    // Step 3: Proposer can still cancel because only proposer equality is checked
```

```

    _mockAndExpect(SAFE, abi.encodeWithSelector(ISafe.nonce.selector),
        abi.encode(1));
    _mockAndExpect(SAFE, abi.encodeWithSelector(ISafe.getTransactionHash.selector),
        abi.encode(bytes32(0)));

    vm.prank(_proposer);
    vm.expectEmit(address(canonGuard));
    emit ICanonGuard.EnqueueTransactionCancelled(_actionsBuilder, _proposer,
        bytes32(0));
    canonGuard.cancelEnqueuedTransaction(_actionsBuilder);

    // Mapping and queue cleared
    (address __proposer, bytes memory __actionsData, uint256 _executableAt, uint256
        _expiresAt, bool _isPreApproved) =
        canonGuard.transactionsInfo(_actionsBuilder);
    assertEq(__actionsData, bytes(''));
    assertEq(_executableAt, 0);
    assertEq(__proposer, address(0));
    assertEq(_expiresAt, 0);
    assertEq(_isPreApproved, false);

    assertEq(canonGuard.getQueuedActionBuilders().length, 0);
}

```

Impact

A removed owner can disrupt governance by cancelling pending transactions they proposed before removal.

Code Snippet

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/blob/main/canon-guard/src/contracts/CanonGuard.sol#L356>

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/blob/main/canon-guard/src/contracts/CanonGuard.sol#L187>

Tool Used

Manual Review

Recommendation

At cancellation time, re-validate that the caller is a current Safe owner. Options include:

- Require `IOwnerManager.isOwner(msg.sender) == true` in addition to `msg.sender == proposer`.
- Alternatively, allow any current Safe owner to cancel queued transactions, or restrict cancellation to the Safe itself (via `isSafe`).

Discussion

Oxng

I see this as a nice to have, but definitely not a medium. In fact, not certain why this wouldn't be an informational.

My rationale is, if you imagine how the real scenario would play out, the fact that a compromised owner can cancel a queued transaction doesn't add any gravity to the worst case scenario.

Let's say there's critical transaction queued up that honest owner Alice proposed. Let's say Alice gets compromised, but the malicious user behind the compromise decides to remain dormant. The other owners approve Alice's proposal, and only then, right before execution, the malicious user cancels it, and forces the other owners to requeue it and to remove the owner.

For that scenario, where the malicious user doesn't give them room to do anything, Alice remains an owner during the attack.

Now, let's think about the scenario where the malicious user does show Alice is compromised while the transaction is queued, which I understand is what you are describing. The moment the compromise is revealed, the owners will queue a transaction to remove the compromised owner. Until that transaction is executed, the compromised owner can cancel the queued transaction at any time. The owners will be aware of this,

and requeue it themselves through a new builder, or if it's critical, they can try to race to trigger the emergency mode.

However, at that point, it's not about the owner cancelling after being removed, but about an owner being compromised at the worst possible time while being the proposer of a critical transaction with a tight deadline.

And there's not much to do about this.

Can you come up with a realistic scenario of impact that justifies the severity, or a counterargument to my rationale?

defsec

Actually, the severity is pushed at the beginning of engagement, therefore we can change it.

The analysis is correct. The compromised owner cancelling after removal does not add meaningful gravity to the worst case scenario. During the removal process which takes 1-7 days the compromised owner is still a Safe owner and can already cancel at any time. Once other owners know Alice is compromised they are already in defensive mode and monitoring for cancellations. Whether Alice can cancel for 7 days or 9 days does not change the response which is to immediately re-queue the transaction.

The only argument I could make is about principles. Removed owners having any residual privilege violates the principle of least privilege. From a pure security architecture perspective once an entity is removed from authorization they should have zero capabilities. This is about correctness and consistency not about exploitable impact.

0xng

The only argument I could make is about principles. Removed owners having any residual privilege violates the principle of least privilege. From a pure security architecture perspective once an entity is removed from authorization they should have zero capabilities. This is about correctness and consistency not about exploitable impact.

Thank you for the reply! Completely agree with what you said there ☑ We will fix it.

0xng

Answer: Confirmed. We will fix this.

defsec

Fix is confirmed.

Issue L-2: Zero cap configuration permanently disables token transfers [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/35>

Summary

The CappedTokenTransfersHub constructor allows setting `cap[token] = 0` for any token without validation. When a token's cap is set to zero, the cap check `totalSpent[_token] > cap[_token]` becomes `totalSpent > 0`, which means any non-zero transfer will be rejected. Since there's no way to update caps after deployment, a token accidentally configured with `cap = 0` becomes permanently unusable for transfers through this hub.

Vulnerability Detail

The constructor in `CappedTokenTransfersHub.sol` accepts an array of caps without validating that each cap is greater than zero:

```
// CappedTokenTransfersHub.sol:44-63
constructor(
    address _safe,
    address _recipient,
    address[] memory _tokens,
    uint256[] memory _caps,
    uint256 _epochLength
) SafeManageable(_safe) ActionHub(msg.sender) {
    RECIPIENT = _recipient;
    EPOCH_LENGTH = _epochLength;
    lastEpoch = block.timestamp;

    if (_tokens.length != _caps.length) revert TokensAndCapsLengthMismatch();
    if (_epochLength == 0) revert EpochLengthCannotBeZero();

    for (uint256 i = 0; i < _tokens.length; i++) {
        if (!_tokens.add(_tokens[i])) {
```

```

        revert TokenAlreadyRegisteredInHub(_tokens[i]);
    }
    cap[_tokens[i]] = _caps[i]; // @audit No validation that _caps[i] > 0
}
}

```

When `updateState()` is called, it checks if the total spent exceeds the cap:

```

// CappedTokenTransfersHub.sol:95-99
totalSpent[_token] += _amount;

if (totalSpent[_token] > cap[_token]) {
    // If cap = 0, this becomes: totalSpent > 0
    // ANY non-zero transfer will revert!
    revert CapExceeded();
}

```

Impact

Any token configured with `cap = 0` becomes permanently unusable for transfers through the hub.

Code Snippet

```

// CappedTokenTransfersHub.sol:58-63
for (uint256 i = 0; i < _tokens.length; i++) {
    if (!_tokens.add(_tokens[i])) {
        revert TokenAlreadyRegisteredInHub(_tokens[i]);
    }
    cap[_tokens[i]] = _caps[i];
}

```

Tool Used

Manual Review

Recommendation

Add validation in the constructor to ensure all caps are greater than zero.

Discussion

0xng

Answer: Confirmed. No harm in adding the check.

defsec

The fix is confirmed.

Issue L-3: Multiple transactions that involve sending native ETH can never be executed in a batch [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/36>

Summary

`executeTransactions` and `_execSafeTransaction` reuse the same `msg.value` for multiple Safe calls, forwarding the full ETH amount multiple times. This means that if there are 2 executable transactions which are supposed to send ETH, the 2nd one will always revert since the 1st call would have consumed the entire available value. This can cause problems in the case where the second action is dependent on the first one, and both actions are time sensitive.

Vulnerability Detail

In `executeTransactions`, the loop repeatedly calls `_execSafeTransaction`, which forwards `msg.value` to `SAFE.execTransaction{value: msg.value}(...)` without decrementing or splitting it. If multiple transactions are executed in a batch, each iteration attempts to forward the same ETH amount again.

```
function executeTransactions(address[] memory _actionsBuilders) external payable {
    ...
    for (uint256 i = 0; i < _actionsBuilders.length; ++i) {
        _execSafeTransaction(_actionsBuilders[i], ...);
    }
}
```

```
function _execSafeTransaction(...) internal {
    ...
    SAFE.execTransaction{value: msg.value}(...);
}
```

```
}
```

In the second call, the safe will call on the `MultiSendCallOnly` which will attempt to forward value that has been consumed which will revert with an `Out of Gas` error.

Impact

Multiple transactions that involve sending native ETH can never be executed in a batch. If subsequent action are dependent on previous ones, and all actions are time sensitive (ie they either need to be executed in a batch or not at all), this can cause operational problems.

Code Snippet

```
/**
 * @notice Internal function to execute a Safe transaction
 * @dev Uses the Safe's execTransaction function
 * @param _multiSendData The encoded MultiSend data
 * @param _signatures The signatures for the transaction
 */
function _execSafeTransaction(bytes memory _multiSendData, bytes memory
    _signatures) internal {
    // @audit msg.value is reused on batch execution
    SAFE.execTransaction{value: msg.value}({
        to: MULTI_SEND_CALL_ONLY,
        value: 0, // Value must be 0 for delegatecall operations
        data: _multiSendData, // if data is bytes(0), the call still succeeds so no
            action txs succeed as expected
        operation: Enum.Operation.DelegateCall,
        safeTxGas: 0,
        baseGas: 0,
        gasPrice: 0,
        gasToken: address(0), // only relevant if gasprice > 0
        refundReceiver: payable(address(0)), // only relevant if gas price is >0
        signatures: _signatures
    });
}
```



```
}
```

Tool Used

Manual Review

Recommendation

1. For each queued `actionsBuilder` compute `requiredValue = sum(action.value for action in actions)`.
2. Require that the caller-supplied `msg.value` (or `CanonGuard` remaining balance) covers the sum of all `requiredValues` across the batch.
3. Forward only `requiredValue` to `SAFE.execTransaction{value: requiredValue}` for that iteration.
4. Keep track of remaining value locally and decrement it by `requiredValue` each iteration.

Discussion

Oxng

Confirmed. We will discuss possible approaches, along with the recommendation.

One more thing, after reviewing the severity criteria of Sherlock, we believe this qualifies as a “high-low” over a medium. The reasoning is that it doesn’t render the contract useless and it should not lead to loss of funds but to a revert.

lpetroulakis

Confirmed. We will discuss possible approaches, along with the recommendation. One more thing, after reviewing the severity criteria of Sherlock, we believe this qualifies as a “high-low” over a medium. The reasoning is that it doesn’t render the contract useless and it should not lead to loss of funds but to a revert.

Agreed. This can be considered borderline medium hence can be downgraded to Low without contest on Sherlock’s end.

Issue L-4: Short-Delay transactions may expire before dependent Long-Delay transactions increasing operational overhead for queuing [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/37>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

If a team plans to execute two dependent actions, one requiring a long delay (not pre-approved) and one requiring a short delay (pre-approved), the short-delay transaction may expire before the long-delay transaction becomes executable. This creates a sequencing problem where both cannot be queued together, increasing operational complexity.

Vulnerability Detail

CanonGuard enforces per-transaction execution and expiry windows based on pre-approval status:

```
uint256 _txExecutionDelay = _actionIsPreApproved ? SHORT_TX_EXECUTION_DELAY :
    LONG_TX_EXECUTION_DELAY;
transactionsInfo[_actionsBuilder] = TransactionInfo({
    executableAt: block.timestamp + _txExecutionDelay,
    expiresAt: block.timestamp + _txExecutionDelay + TX_EXPIRY_DELAY,
    ...
});
```

If `SHORT_TX_EXECUTION_DELAY` (e.g., 1 day) and `LONG_TX_EXECUTION_DELAY` (e.g., 7 days), the short-delay transaction expires after 1d + expiry, while the dependent long-delay

transaction only becomes executable at 7d. Thus, when queued together, the short-delay transaction's expiry window can close before the long-delay transaction can be executed.

Impact

Teams using the Canon Guard must re-queue dependent short-delay transactions manually near the long-delay execution time which increases operational overhead

Code Snippet

In `CanonGuard::_queueTransaction`

```
uint256 _txExecutionDelay = _actionIsPreApproved
    ? SHORT_TX_EXECUTION_DELAY
    : LONG_TX_EXECUTION_DELAY;

transactionsInfo[_actionsBuilder] = TransactionInfo({
    executableAt: block.timestamp + _txExecutionDelay,
    expiresAt: block.timestamp + _txExecutionDelay + TX_EXPIRY_DELAY,
    ...
});
```

Tool Used

Manual Review

Recommendation

Document the issue thoroughly for teams to avoid unexpected expiries.

Discussion

Oxng

Acknowledged. The UI user experience we are building aims to tackle this issue, and in addition it you could always enqueue it again. We expect Canon Guard to only be used

Issue L-5: Incorrect transaction hash emitted when canceling future queued transactions [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/38>

Summary

`cancelEnqueuedTransaction` incorrectly assumes that the transaction being canceled corresponds to the current `Safe` nonce, even though the `_actionsBuilder` parameter can point to any queued (future) transaction. This causes the calculated `_safeTxHash` that is emitted to be inaccurate for non-current nonces.

Vulnerability Detail

The `cancelEnqueuedTransaction` function calculates the transaction hash using the current `Safe` nonce, regardless of which queued transaction is being canceled:

```
bytes memory _multiSendData = _buildMultiSendData(_actions);
bytes32 _safeTxHash = _getSafeTransactionHash(_multiSendData, SAFE.nonce());
```

However, the `_actionsBuilder` being canceled can correspond to a transaction intended for a future nonce. Since `SAFE.nonce()` reflects the current pending execution index, the resulting hash does not match the actual transaction's true hash (which depends on its intended nonce).

Impact

Off chain indexers may incorrectly record the canceled transaction, since the emitted `_safeTxHash` will not match the real queued transaction's hash which can cause confusion

Code Snippet

```
bytes memory _multiSendData = _buildMultiSendData(_actions);  
@> bytes32 _safeTxHash = _getSafeTransactionHash(_multiSendData, SAFE.nonce());  
emit EnqueuedTransactionCancelled(_actionsBuilder, msg.sender, _safeTxHash);
```

Tool Used

Manual Review

Recommendation

Remove the hash calculation altogether since it is not used anywhere else other than the event emission. In the event, only emit the action builder address being cancelled.

Discussion

Oxng

Answer: Confirmed. We will modify or remove the event, and the hash calculation.

Issue L-6: Compromised signer can increase transaction queue until removed, leading to `getQueuedActionBuilders` DOS. [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/39>

Summary

A compromised signer can grief the system by spamming the queue with arbitrary transactions using unique `_actionsBuilder` addresses. This bloats the `__queuedActionBuilders` set, potentially causing `getQueuedActionBuilders()` to revert due to gas exhaustion. Cleaning up those malicious queued transactions to solve the problem will incur non trivial cost.

Vulnerability Detail

Because any Safe owner can call `queueTransaction()`, a single compromised signer can repeatedly queue transactions with unique `_actionsBuilder` addresses.

There is no limit on the size of `__queuedActionBuilders`, and removal only occurs upon transaction execution or explicit cancellation by the original proposer (or `emergencyCaller` in emergency mode). This means a malicious owner can indefinitely increase the set size.

As a result:

- The `getQueuedActionBuilders()` function can revert due to excessive gas usage when iterating over the large set.
- Off-chain systems relying on this call (e.g., dashboards) can fail.
- Even in emergency mode, canceling all queued transactions is costly, requiring one transaction per entry.

Impact

DoS for off-chain systems or UIs using `getQueuedActionBuilders()`.

Code Snippet

Tool Used

Manual Review

Recommendation

Solady's `EnumerableSetLib` allows for a maximum cap size to a set when adding items. Consider implementing this along with a way to remove expired transactions such that the cap is never reached in practice.

From Solady:

```
/// @dev Adds `value` to the set. Returns whether `value` was not in the set.
/// Reverts if the set grows bigger than the custom on-the-fly capacity `cap`.
function add(AddressSet storage set, address value, uint256 cap)
    internal
    returns (bool result)
{
    if (result = add(set, value)) if (length(set) > cap) revert ExceedsCapacity();
}
```

Discussion

0xng

Answer: Confirmed. Although this doesn't affect the functionality of the Safe, we will brainstorm a way for this not to be an issue.

Issue L-7: Outdated Documentation in `cancelEnqueuedTransaction` as it no longer verifies approved hashes [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/40>

Summary

The documentation for `cancelEnqueuedTransaction` states that it should only cancel a transaction if it has no approved hashes. However, this logic has since been removed. The function now derives the Safe transaction hash but never checks for approvals before canceling.

Vulnerability Detail

The function's intended behavior per documentation is:

```
/**
 * @notice Cancels an enqueued transaction
 * @dev Can only be called by the proposer of the transaction, aka a safe signer
 * @dev The transaction must not have any approved hash signers
 * @param _actionsBuilder The actions builder contract address
 */
function cancelEnqueuedTransaction(address _actionsBuilder) external {
```

In the current implementation, however, after computing the Safe transaction hash, no verification against `SAFE.approvedHashes` is performed. The function directly removes the `_actionsBuilder` from the queue, allowing cancellation even if the transaction has already been approved.

Impact

The Safe allows cancellation of approved transactions, contrary to documentation

Code Snippet

Tool Used

Manual Review

Recommendation

Update the documentation to reflect the current intended behavior

Discussion

0xdeadbeef0x

Opening a discussion here as this issue can have some significant impact -

consider a legitimate approved critical action was proposed by a signer and that signer was either malicious or compromised during the timelock. The attacker waits just before execution to cancel it. Since the action is critical, the need to propose and wait for execution could be meaningful.

What do you think?

0xng

Answer: Confirmed. We will fix the documentation. We still believe removing the check of the approval is the right choice since checking a specific nonce can give you a false sense of confidence. With regards to the comment about cancelling a critical transaction just before execution, then:

- We believe it's a very rare edge case
- We don't expect cases like this one to fully disrupt the protocol

In this path, the signer of the multisig has been compromised and the only thing that he can do is cancel a transaction. That sounds like a good worst case.

We know this may not be the optimal solution, but we consider it fine for the first version of canon guard.

Issue L-8: In `getQueuedActionBuilders`, expired transactions cause unbounded gas costs [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/41>

Summary

Expired transactions remain in the `__queuedActionBuilders` `EnumerableSet` indefinitely until explicitly removed via execution or cancellation. The `getQueuedActionBuilders()` function returns all entries in the set, including expired ones, leading to unbounded gas costs that grow over time.

Vulnerability Detail

When transactions expire (`expiresAt <= block.timestamp`), they are not automatically removed from the `__queuedActionBuilders` set. They remain in the set until:

1. Explicitly executed via `executeTransaction()` (which removes them)
2. Explicitly cancelled via `cancelEnqueuedTransaction()` (which removes them)

The `getQueuedActionBuilders()` view function returns all entries in the set without filtering:

```
// CanonGuard.sol:257-260
/// @inheritdoc ICanonGuard
function getQueuedActionBuilders() external view returns (address[] memory
    _queuedActionBuilders) {
    _queuedActionBuilders = __queuedActionBuilders.values(); // @audit Returns
        ALL entries, including expired
}
```

When a new transaction is queued for an expired builder, the expired entry is overwritten

but not removed from the set:

```
// CanonGuard.sol:343-349
// Revert if the transaction is already queued and not expired
if (!__queuedActionBuilders.add(_actionsBuilder)) {
    TransactionInfo memory _queuedTransactionInfo =
        transactionsInfo[_actionsBuilder];
    if (_queuedTransactionInfo.expiresAt > block.timestamp) {
        revert TransactionAlreadyQueued(_actionsBuilder);
    }
    // @audit If expired, overwrites TransactionInfo but doesn't remove from set
    // The set entry remains, causing getQueuedActionBuilders() to return it
}
```

Impact

The `getQueuedActionBuilders()` view function's gas cost grows linearly with the number of expired transactions.

Code Snippet

```
function getQueuedActionBuilders() external view returns (address[] memory
    _queuedActionBuilders) {
    _queuedActionBuilders = __queuedActionBuilders.values();
}
```

Tool Used

Manual Review

Recommendation

Add a function to clean up expired transactions.

Discussion

0xng

Confirmed. This is related to #39. Same answer as in there. We will brainstorm a complete solution to address the queue staffing problem.

defsec

The fix is confirmed.

Issue L-9: Execution through modules can bypass guards. [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/42>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Other than user execution, Safes support execution through modules which can completely bypass CanonGuard protections.

Vulnerability Detail

Execution of transactions through modules (external contracts) are done via `execTransactionFromModule` which completely bypasses signatures from owners.

If a Safe that adopts the canon guard already has a module attached that can execute transactions via `execTransactionFromModule` then the canon guard protection is completely bypassed.

Two scenarios:

1. if the safe that adopts the canon guard already has a module attached that can execute transactions via `execTransactionFromModule` then the cannon guard protection is completely bypassed so at least this should be documented somehow
2. if the safe that adopts the canon guard does not have any modules attached, `enableModule` would need to be called which needs to go through the timelock first and an additional timelock to add a guard to the module if needed, which will introduce a timeframe where there is no guard for the module.

Impact

Potential bypass of guard protections.

Code Snippet

Tool Used

Manual Review

Recommendation

If the goal of the CanonGuard is to disallow all transactions from modules, then it should:

1. Block enableModule transactions from ever being queued or executed
2. Make sure modules are not enabled on existing Safes and hook on calls to them and revert.
3. Document

Discussion

0xng

Answer: Acknowledged. This is an explicit design decision. We will not fix. The users of CanonGuard are expected to be technical enough to understand the risks of introducing a Module, or a FallbackHandler--which are also documented on the Safe's docs.

Issue L-10: Shared lastEpoch causes DoS for same epoch multi-token transfers [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/43>

Description

The CappedTokenTransfersHub contract uses a single shared lastEpoch state variable for all tokens, which creates a critical denial-of-service vulnerability when multiple tokens need to be transferred within the same epoch.

The bug occurs in the updateState function:

```
function updateState(address _token, uint256 _amount) external isSafe {
    // ... epoch calculation ...

    // If we're in a new epoch, reset the spending
    if (_currentEpoch > lastEpoch) {
        delete totalSpent[_token]; // Only this specific token's spending is reset
        lastEpoch = _currentEpoch; // But lastEpoch is updated globally for ALL
                                    tokens
    }

    totalSpent[_token] += _amount;

    if (totalSpent[_token] > cap[_token]) {
        revert CapExceeded();
    }
}
```

Fix

Create a per token lastEpoch map

Discussion

Oxng

Confirmed. We will fix this. Regarding the severity. We also see this one as a "high-low" given this action is not part of the core logic of `CanonGuard`, and also doesn't lead to any loss of funds, but to reverts. In reality, this hub can be redeployed and reapproved.

Still, this is clearly a bug and needs fixing.

Issue L-11: Inefficient bubble sort algorithm with $O(n^2)$ complexity used on every transaction execution [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/45>

Description

The `_sortSigners()` function in `CanonGuard.sol` uses bubble sort with $O(n^2)$ time complexity for both best and worst cases. This inefficient sorting algorithm runs on every transaction execution, consuming unnecessary gas even when the array is already sorted or nearly sorted.

```
function _sortSigners(address[] memory _signers) internal pure {
    uint256 _signersLength = _signers.length;
    address _temp;
    for (uint256 _i; _i < _signersLength; ++_i) {
        for (uint256 _j; _j < _signersLength - _i - 1; ++_j) {
            if (_signers[_j] > _signers[_j + 1]) {
                _temp = _signers[_j];
                _signers[_j] = _signers[_j + 1];
                _signers[_j + 1] = _temp;
            }
        }
    }
}
```

Since Safe owners are typically already stored in sorted order and change infrequently, using insertion sort would provide $O(n)$ best-case complexity for already-sorted arrays while maintaining $O(n^2)$ worst-case. Additionally, caching the sorted owners list when it doesn't change between transactions would eliminate redundant sorting operations entirely, significantly reducing gas costs for frequent transaction executions.

Discussion

0xng

Confirmed. Will fix.

Issue L-12: Single point of failure in emergency mode [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/46>

This issue has been acknowledged by the team but won't be fixed at this time.

Description

In the current design, the emergency mode only allows the `emergencyCaller` to execute transactions. However, if this address is compromised, the safe would basically be rendered useless as only the `emergencyCaller` would be able to trigger the `unsetEmergencyMode()` call.

This design is fundamentally different from the design principles of safe

Recommendation

Since the emergency mode is meant for temporary emergency situations, which do not involve complete compromise of the owners' private keys, a better design would be to completely remove the emergency trigger and caller and implement an emergency duration system instead.

Discussion

Oxng

Confirmed. We are aware of this. CanonGuard was designed with the idea that the `emergencyCaller` can't be compromised. However, we will brainstorm either a solution, or a best practice for teams to accomplish this. We have some ideas in mind, and will share soon!

OxGorilla

We decided not to implement any solution to this issue, since we consider it to be completely external to Canon Guard.

For example, if you believe your `emergencyCaller` may be lost in the future, you should be cautious and add a dead man switch mechanism to your emergency caller msg.

We agree on the concern, just we do think this should be part of educating the user and possibly the UX, but not the core.

Issue L-13: CappedTokenTransferHub _recipient not checked for zero address on creation [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/47>

Summary

CappedTokenTransfersHub does not validate `_recipient != address(0)`. A zero recipient can lead to ERC20 transfers to the zero address

```
constructor(
    address _safe,
    address _recipient,
    address[] memory _tokens,
    uint256[] memory _caps,
    uint256 _epochLength
) SafeManageable(_safe) ActionHub(msg.sender) {
    RECIPIENT = _recipient;
    EPOCH_LENGTH = _epochLength;
    lastEpoch = block.timestamp;

    if (_tokens.length != _caps.length) revert TokensAndCapsLengthMismatch();
    if (_epochLength == 0) revert EpochLengthCannotBeZero();

    for (uint256 i = 0; i < _tokens.length; i++) {
        if (!__tokens.add(_tokens[i])) {
            revert TokenAlreadyRegisteredInHub(_tokens[i]);
        }
        cap[_tokens[i]] = _caps[i];
    }
}
```

Recommendation

Add a check like:

```
if (_recipient == address(0)) revert RecipientAddressZero();
```

Discussion

0xng

Confirmed. We will add the check.

Issue L-14: Approved hash check not aligned with Safe interface description [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/48>

Summary

`_getApprovedHashSigners` checks `SAFE.approvedHashes(owner, hash) == 1`. Safe's `approvedHashes` interface treat any non-zero value as approval. Equality to 1 is brittle and can fail if a future or custom Safe increments this value differently, causing valid approvals to be ignored.

Vulnerability Detail

CanonGuard enumerates owners and treats an owner as approved only when `approvedHashes(owner, hash) == 1`:

```
if (SAFE.approvedHashes(_safeOwner, _safeTxHash) == 1) {
    _tempSigners[_approvedHashSignersCount] = _safeOwner;
    ++_approvedHashSignersCount;
}
```

Safe's interface is "non-zero means approved". Tight equality unnecessarily ties CanonGuard to a specific implementation detail of the current dependency commit. If the value differs (e.g., increments), owners that approved may be excluded, making execution revert for lack of signatures.

Here is Safe's interface description: <https://github.com/safe-fndn/safe-smart-account/blob/main/contracts/interfaces/ISafe.sol#L207-L213>

```
* @notice Returns a non-zero value if the `messageHash` is approved by the `owner`.
* @param owner The owner address that may have approved a hash.
* @param messageHash The hash of message to check.
* @return An integer denoting whether or not the hash is approved by the owner.
*         A non-zero value indicates that the hash is approved.
```



```
*/  
function approvedHashes(address owner, bytes32 messageHash) external view returns  
    (uint256);
```

Impact

Valid approvals may be ignored, preventing execution in future versions of Safe

Code Snippet

Tool Used

Manual Review

Recommendation

Loosen the predicate to accept any non-zero:

```
if (SAFE.approvedHashes(_safeOwner, _safeTxHash) != 0) {  
    // count as approved  
}
```

Discussion

0xng

Confirmed. We will change the check.

Issue L-15: Unused error definition [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/49>

Summary

The error `NotAnActionsBuilder()` is defined in the `ICanonGuard` interface but is never used in the implementation.

Vulnerability Detail

The `queueTransaction()` function accepts any address without validating that it implements the `IActionsBuilder` interface. This leads to poor error messages when invalid addresses are passed, as the failure occurs later during the `getActions()` call with unclear revert reasons.

Impact

Optimization

Code Snippet

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/blob/main/canon-guard/src/interfaces/ICanonGuard.sol#L142>

Tool Used

Manual Review

Recommendation

Delete an unused error from interface.

Discussion

0xng

Confirmed. We will remove this from the interface.

defsec

Fix is confirmed.

Issue L-16: No revert ERC20 tokens will fill cap on failure [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/52>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

In `CappedTokenTransfers`, the hub state is updated before the token transfer, and the ERC20 transfer return value is not enforced. For tokens that return `false` on failure (non-reverting), the `Safe` call succeeds but no tokens move, while the hub's `totalSpent` increases. This permanently consumes cap without delivering funds, DoSing transfers until the cap resets.

Vulnerability Detail

The builder constructs two actions: first `updateState` on the hub, then `IERC20.transfer`:

```
// First action: update state
_actions[0] =
  Action({target: HUB, data: abi.encodeCall(ICappedTokenTransfersHub.updateState,
    (TOKEN, AMOUNT)), value: 0});

// Second action: transfer
_actions[1] = Action({target: TOKEN, data: abi.encodeCall(IERC20.transfer,
  (RECIPIENT, AMOUNT)), value: 0});
```

`Safe/MultiSend` do not enforce ERC20 boolean return values. For tokens that signal failure by returning `false` (without reverting, see: <https://github.com/d-xo/weird-erc20?tab=readme-ov-file#no-revert-on-failure>), the second call is considered successful at the EVM level. Result: `totalSpent[TOKEN]` is incremented, but no tokens move.

Since cap accounting is already incremented, further transfers for that token may revert with `CapExceeded` until the next epoch.

Impact

A token that returns `false` (non-reverting failure) can permanently consume cap without delivering funds

Code Snippet

Tool Used

Manual Review

Recommendation

perform the transfer first, then `updateState` after verifying that the balance has changed.

Discussion

0xng

Confirmed/Acknowledged. We are aware of this. Handling return values, and responding to them in general was not straightforward. For one, you need to cache the previous balance, then do the transfer, then grab the cached previous balance to properly check for this. We will revisit it though, and see if we can brainstorm a more elegant way to handle these cases. In general, we believe that it will be rare due to the expected technical knowledge and the simulations and inspections the teams should run for they to run into these cases.

0xAustrian

Update: After revisiting the code, we have decided to keep it as is, prioritizing simplicity.

Issue L-17: Immutable recipient in CappedTokenTransfersHub limits operational flexibility [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2025-10-velodrome-canon-guard-nov-3rd/issues/54>

This issue has been acknowledged by the team but won't be fixed at this time.

Description

The CappedTokenTransfersHub contract uses an immutable RECIPIENT address that is set once during deployment and cannot be changed.

This design severely limits operational flexibility, as the Safe may need to redirect capped transfers to different recipients.

Each time a new recipient is required, a whole new CappedTokenTransfersHub instance has to be deployed.

Recommendation

If the team still wants to keep the current design, I can suggest a more efficient way to avoid having to deploy CappedTokenTransfersHub for each new recipient.

1. Instead of a single recipient, implement a list of permissioned recipients.
2. Add another function to CappedTokenTransfersHub which has the `isSafe` modifier, and allows the Safe to add a recipient to the allow list of recipients.

This way the decision to add a new recipient would still take the Long path but would be more efficient as it requires one less step (contract deployment)

Discussion

Oxng

Acknowledged. Won't fix. By design we prefer immutability.

lpetroulakis

Acknowledged - no fix to be implemented at the time.

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.